Fonctions

Définir ses propres fonctions

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1

Définir ses propres fonctions

Identifier, nommer, puis réutiliser un code d'utilité générale qui prend des entrées et produit un résultat en sortie

- Motivation
- Définition de fonction : nommage
- Paramètres et valeur de retour
- Définition d'une fonction en Python
- Appeler une fonction
- Ordre des éléments : définition puis appels
- Variables propres à la fonction
- Chaînage de fonctions

2

Exercices

Maximum en valeur absolue, chaînage de fonctions, aires de rectangles

Motivation

On identifie un morceau de code qu'on va vouloir réutiliser plusieurs fois. Il faut que ce morceau de code ait une utilité générale, sinon il n'y aurait pas de raison de le réutiliser.

On va **définir** une fonction une fois dans le programme, puis utiliser (on dit **appeler**) cette fonction plusieurs fois.

Gains :

- On n'a pas besoin de répéter du code.
- On peut facilement modifier la fonction (par exemple pour la corriger si on trouve un problème dedans) au lieu de devoir faire la modification partout.
- Quand on est en train d'écrire un programme complexe, on n'a pas besoin de penser à tous les détails de comment les fonctions font leur travail.

Définition de fonction : nommage

Une fois qu'on a identifié un morceau d'algorithme d'utilité générale, il faut lui trouver un nom :

- qui permet immédiatement de comprendre ce que fait la fonction juste en regardant le nom (sans avoir à regarder tout le code à l'intérieur) ;
- qu'on peut facilement prononcer ;
- qui obéit aux mêmes règles que les noms de variables : le nom d'une fonction doit commencer par une lettre et ne pas contenir d'espace.

```
f
ma fonction
maximum
f2
2f
```

```
valeur absolue
valeurabsolue
valeur_absolue
valeurAbsolue
```

En rouge : refusé par Python. En orange : accepté, mais mal choisi.

Paramètres et valeur de retour

On a dit qu'une fonction avait une utilité générale : par exemple, une fonction qui calcule le volume d'une sphère peut être utilisée pour calculer le volume d'un microplastique, d'un ballon de foot, de la Lune, du Soleil. On a besoin de deux mécanismes :

- un mécanisme pour pouvoir donner à la fonction les informations dont elle a besoin. Dans cet exemple, c'est le rayon de la sphère ;
- un mécanisme pour que la fonction puisse rendre le résultat qu'elle a produit, pour qu'on puisse ensuite l'utiliser (par exemple pour l'afficher, ou pour le comparer avec quelque chose).

Paramètres et valeur de retour de fonctions que vous connaissez déjà

Description	Fonction	Nom(s) du (des) paramètre(s)	Type de la valeur de retour
Convertit une chaîne de caractères en entier			
Convertit une chaîne de caractères en nombre décimal			
Renvoie la longueur d'une liste			
Calcule la somme d'une liste			
Demande une valeur à la personne qui utilise le programme			
Affiche des informations			

Paramètres et valeur de retour de fonctions que vous connaissez déjà

Description	Fonction	Nom(s) du (des) paramètre(s)	Type de la valeur de retour
Convertit une chaîne de caractères en entier	`int`	`chaine`, `s`	nombre entier ou erreur
Convertit une chaîne de caractères en nombre décimal	`float`	`chaine`, `s`	nombre décimal ou erreur
Renvoie la longueur d'une liste	`len`	`l`, `lst`, `liste`	nombre entier
Calcule la somme d'une liste	`sum`	`l`, `lst`, `liste`	nombre entier ou décimal
Demande une valeur à la personne qui utilise le programme	`input`	`message`	chaîne de caractères
Affiche des informations	`print`	`s1`, `s2`, `s3`, `s4`	ne retourne rien. Ce genre de fonction est une *procédure*

Définition d'une fonction en Python


```
def nom_de_la_fonction(param1, param2, ...):
    ...
    ...
    return valeur_de_retour

suite du programme
```

- Le mot clé `def` introduit la définition d'une fonction.
- Les noms de paramètres suivent les mêmes règles que les noms de variables et de fonction.
- La ligne finit avec `:`
- Les lignes suivantes sont indentées (décalées depuis la marge).
- Le bloc indenté est appelé *corps de la fonction*.
- Dans le corps de la fonction, il y a habituellement un `return` avec la valeur calculée par la fonction.
- La définition de la fonction est terminée quand le code revient à la marge.

Exemple 1 : fonction nombre au cube


```
1 def cube(x):
2     return x * x * x
```

```
>>> print(cube(2))

>>> x = cube(3)
>>> print(x)

>>> print(cube(3) + cube(4))
```

Exemple 1 : fonction nombre au cube


```
1 def cube(x):
2     return x * x * x
```

```
>>> print(cube(2))
8
>>> x = cube(3)
>>> print(x)
27
>>> print(cube(3) + cube(4))
91
```

Exemple 2 : fonction maximum de deux nombres

Fonction correcte avec un seul `return`

```
def maximum(a, b):
    if a > b:
        res = a
    else:
        res = b
    return res
```

Exemple 2 : fonction maximum de deux nombres

Fonction correcte avec un seul `return`

```
1 def maximum(a, b):
2     if a > b:
3         res = a
4     else:
5         res = b
6     return res
```

Fonction équivalente avec deux `return`

```
1 def maximum(a, b):
2     if a > b:
3         return a
4     else:
5         return b
```

On peut utiliser plusieurs fois le mot clé `return`, qui va immédiatement finir d'exécuter la fonction pour retourner le résultat.
Cela permet souvent de simplifier l'écriture de la fonction.

Appeler une fonction

Définition de la fonction

```
def la_fonction(param1, param2):
    ...
    ...
    return valeur_de_retour
```

Appel de la fonction

```
x = la_fonction(argument1, argument2)
```

On écrit simplement le nom de la fonction, puis, entre parenthèses, on fournit autant d'*arguments* qu'il y a de *paramètres* dans la définition de la fonction.

On peut faire ce qu'on veut de la valeur retournée par la fonction : ici, on la stocke dans une variable x ; on pourrait aussi l'afficher, ou l'utiliser dans un autre appel de fonction.

Ordre des éléments : définition puis appels

Il faut toujours *définir* une fonction **avant** de l'*appeler*.

```
def somme(liste):
    s = 0
    for e in liste:
        s = s + e
    return s

print(somme([4, 8, 11]))
print(somme([4, 2]))
grand_total = somme([6, 7, 9]) + somme([8, 20])
```

Variables propres à la fonction

Un programme qui définit et utilise une fonction

```
1  def somme(liste):
2      s = 0
3      for e in liste:
4          s = s + e
5      return s
6
7  s = input("Est-ce que c'est super ? ")
8  print(somme([4, 8, 11]))
9  print(s)
```

Activité · 3 min

Que va afficher ce programme quand on l'exécute ?

Variables propres à la fonction

Un programme qui définit et utilise une fonction

```
1 def somme(liste):
2     s = 0
3     for e in liste:
4         s = s + e
5     return s
6
7 s = input("Est-ce que c'est super ? ")
8 print(somme([4, 8, 11]))
9 print(s)
```

Activité

Que va afficher ce programme quand on l'exécute ?

Les variables définies dans le corps de la fonction vivent dans la fonction uniquement pendant que la fonction s'exécute.
Il n'y a aucun risque de collision entre des variables qu'on définit dans une fonction et les autres variables du programme.

On peut considérer que le *corps* de la fonction est un secret qui ne concerne que la personne qui écrit (définit) la fonction ; la personne qui appelle la fonction n'a besoin de connaître que le nom de la fonction, et le type, le nombre et la signification des paramètres qu'elle attend.

```
Est-ce que c'est super ? Oui
23
Oui
```

Combinaison / chaînage de fonctions

Dans les deux lignes de code, on demande une valeur à la personne qui utilise le programme, puis on la convertit en nombre entier :

```
réponse = input("âge : ")
âge = int(réponse)
```

On peut arriver au même résultat en une seule ligne, en chaînant les deux appels de fonctions :

```
âge = int(input("âge : "))
```

Attention : il faut prendre l'habitude de lire ce genre de combinaison **de l'intérieur vers l'extérieur des parenthèses** :

- d'abord, on fait un `input` ;
- puis le résultat du `input` est transformé en entier avec `int`.

Un autre exemple de chaînage :

```
def inc(x):
    return x + 1
```

```
>>> inc(0)
1
>>> inc(inc(0))
2
>>> inc(inc(inc(inc(inc(0)))))
5
```

Sommaire

1

Définir ses propres fonctions

Identifier, nommer, puis réutiliser un code d'utilité générale qui prend des entrées et produit un résultat en sortie

- Motivation
- Définition de fonction : nommage
- Paramètres et valeur de retour
- Définition d'une fonction en Python
- Appeler une fonction
- Ordre des éléments : définition puis appels
- Variables propres à la fonction
- Chaînage de fonctions

2

Exercices

Maximum en valeur absolue, chaînage de fonctions, aires de rectangles

Exercice 1 : maximum en valeur absolue

Exercice · 10 min

On aimerait afficher la plus grande valeur absolue de deux nombres.

Par exemple : −10 et 4 → 10 −4 et 3 → 4 −4 et −3 → 4

Définir deux fonctions pour réaliser cette tâche :

- une qui renvoie la valeur absolue d'un nombre.
  Indice : on peut changer le signe d'un nombre en le multipliant par −1 ;
- une qui renvoie le maximum de deux nombres.

Combiner ces deux fonctions pour calculer et afficher la réponse pour les 3 exemples donnés plus haut.

Exercice 1 : maximum en valeur absolue

Afficher la plus grande valeur absolue de deux nombres, avec une fonction qui renvoie la valeur absolue d'un nombre et une qui renvoie le maximum de deux nombres.
Par exemple : −10 et 4 → 10 −4 et 3 → 4 −4 et −3 → 4

```python
def valeur_absolue(x):
    if x < 0:
        return x * -1
    else:
        return x

def maximum(a, b):
    if a > b:
        return a
    else:
        return b

print(maximum(valeur_absolue(-10), valeur_absolue(4)))
print(maximum(valeur_absolue(-4), valeur_absolue(3)))
print(maximum(valeur_absolue(-4), valeur_absolue(-3)))
```

Le programme affiche :

```
10
4
4
```

Exercice 2 : chaînage de fonctions

Étant donné ces 4 fonctions :

```
def plus(a, b):
    return a + b

def moins(a, b):
    return a - b

def fois(a, b):
    return a * b

def neg(a):
    return -a
```

On peut écrire des expressions sans utiliser les opérateurs + - *

Par exemple : `-4 * 12` peut s'écrire : `fois(neg(4), 12)`

Exercice · 8 min

Réécrire les expressions suivantes en remplaçant les opérateurs + - * par l'une des 4 fonctions de gauche :

a) `1 + 3 + 5`
b) `- (10 + 4)`
c) `2 + 3 * 4 + 5`

Il n'est pas question de modifier les fonctions de gauche ou d'en ajouter : il faut utiliser ce qui est là.

Dans Moodle, on vous donne un programme à compléter (que vous rendez complété dans Moodle).

Exercice 2 : chaînage de fonctions

Réécrire les expressions en remplaçant les opérateurs `+ - *` par les fonctions `plus`, `moins`, `fois` et `neg` :

	Expression	Avec les fonctions	Valeur
a)	`1 + 3 + 5`	`plus(plus(1, 3), 5)`	9
b)	`- (10 + 4)`	`neg(plus(10, 4))`	−14
c)	`2 + 3 * 4 + 5`	`plus(plus(2, fois(3, 4)), 5)`	19

Exercice 3 : aires de rectangles

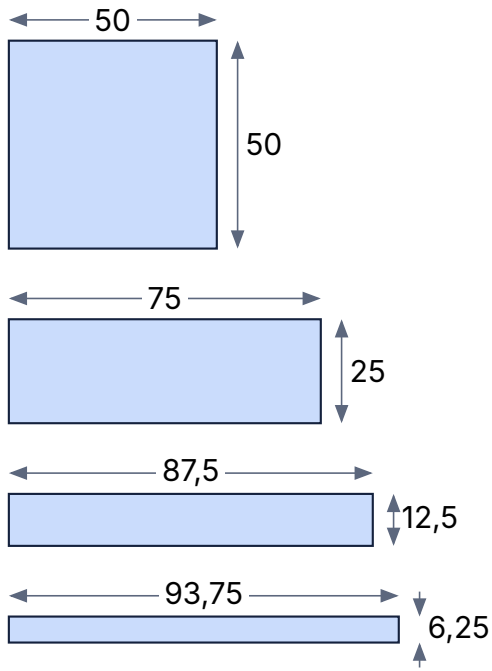
Exercice · 15 min

a) Définir une fonction qui calcule l'aire d'un rectangle. Choisir un bon nom pour la fonction et le(s) paramètre(s).

b) Utiliser cette fonction pour calculer et afficher l'aire des rectangles (4, 3), (3, 3) et (3, 4).

c) Utiliser cette fonction pour calculer et afficher l'aire de tous les rectangles de longueur 10 et de largeurs : 0, 10, 20, 30, 40, ..., 100.

d) Utiliser cette fonction pour calculer et afficher l'aire de tous les rectangles qui ont un périmètre de 200. À droite, quelques exemples.



Exercice 3 : aires de rectangles

a) Définir une fonction qui calcule l'aire d'un rectangle, puis l'utiliser pour afficher l'aire : b) des rectangles (4, 3), (3, 3) et (3, 4) ; c) des rectangles de longueur 10 et de largeurs 0, 10, 20, ..., 100 ; d) des rectangles qui ont un périmètre de 200.

```
def aire_rectangle(longueur, largeur):  # a)
    return longueur * largeur

print(aire_rectangle(4, 3))      # b)
print(aire_rectangle(3, 3))
print(aire_rectangle(3, 4))

largeur = 0                      # c)
while largeur <= 100:
    print(aire_rectangle(10, largeur))
    largeur = largeur + 10

longueur = 0                     # d) longueur + largeur = 100
while longueur <= 100:
    print(aire_rectangle(longueur, 100 - longueur))
    longueur = longueur + 10
```